START DATA MANAGEMENT

FUNÇÕES EM PYTHON

5

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

LISTA DE FIGURAS

Figura 1 – Builtins 6
Figura 2 – Help(dir) 6
Figura 3 – Help(print) 7
Figura 4 – Type() Error 7
Figura 5 – Saída da Saudação com número de parâmetros variáveis 14
Figura 6 – Como usar a função lambda 15
Figura 7 – Como usar a função lambda com desvios condicionais 16

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Uso das funções len e print 7
Código-fonte 2 – Algoritmo de velocidade média sem função 9
Código-fonte 3 – Criando uma nova função em Python 9
Código-fonte 4 – Colocando o algoritmo dentro de uma função em Python..... 10
Código-fonte 5 – Chamada para a função criada em Python 10
Código-fonte 6 – Modificando a função para receber parâmetros 11
Código-fonte 7 – Modificando a função para receber parâmetros 11
Código-fonte 8 – Modificando a função para retornar um valor 12
Código-fonte 9 – Modificando a chamada da função que retorna um valor..... 13
Código-fonte 10 – Função de saudação com número de parâmetros variáveis..... 13
Código-fonte 11 – Estrutura básica do lambda 14
Código-fonte 12 – Soma de 2 números com lambda 15
Código-fonte 13 – Estrutura básica do lambda 15
Código-fonte 14 – Positivo ou Negativo com lambda 16
Código-fonte 15 – Estrutura básica do map 16
Código-fonte 16 – Map com função dobro 17
Código-fonte 17 – Map com função lambda dobro 17
Código-fonte 18 – Módulo calc.py 18
Código-fonte 19 – Importação do módulo calc.py 18
Código-fonte 20 – Uso da função soma do módulo calc.py 19
Código-fonte 21 – Importação de funções específicas do módulo calc.py 19
Código-fonte 22 – Chamada de uma função especificada na importação do módulo calc.py 19
Código-fonte 23 – Importação de todas as funções do módulo calc.py..... 19
Código-fonte 24 – Calculando o preço da pizza 20
Código-fonte 25 – Calculando o preço da pizza 21

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

SUMÁRIO

1 FUNÇÕES 5
1.1 Tem função aí? __builtins__ 5
1.2 Minha primeira função 9
1.3 O poder dos parâmetros 10
1.4 Alô!? Me responde função! - retorno de valores 12
1.5 1, 2, 3, N parâmetros 13
2 FUNÇÕES INLINE 14
2.1 É patins? Grego? Funções INLINE ou Lambda 14
2.2 MAP 16
3 A FRONTEIRA FINAL 17
3.1 Criando o módulo 17
3.2 Utilizando o módulo 18
4 HANDS ON: HORA DA PRÁTICA 20
4.1 Função é essencial 20
4.1.1 Vai de pizza? 20
4.1.2 Viagem intergaláctica 21
REFERÊNCIAS 22

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

1 FUNÇÕES

Você trabalha em uma empresa de transportes e consegue criar um algoritmo para calcular a velocidade média que os caminhões da empresa desenvolvem para cumprir um determinado trajeto. Entretanto, existe uma pegadinha: você precisa usar esse mesmo algoritmo em diferentes partes do seu sistema!

A solução mais tradicional seria utilizar o famoso “CTRL+C, CTRL+V” e copiar seu algoritmo em todos os lugares em que for necessário, correto? Mas hoje você vai aprender porque essa é uma péssima solução.

Os dois principais problemas são: o seu programa vai ter muito mais linhas do que ele precisaria ter, e a manutenção do código será fortemente dificultada, pois cada alteração que seu algoritmo tiver, terá de ser replicada em todas as cópias.

Para resolver esse problema de forma inteligente, existem as funções, que podem ser consideradas “micro” programas que têm uma única função específica e podem ser convocadas quando necessárias.

Vamos entender melhor?

1.1 Tem função aí? __builtins__

Em Python, todos os símbolos (variáveis, classes, funções etc.) necessários precisam ser importados para estarem disponíveis, e até agora ainda não vimos como fazer isso. Porém, existe um módulo "embutido" da linguagem chamado ***__builtins__***, que é importado automaticamente. É nesse módulo que os tipos de dados mais básicos são definidos e um grande conjunto de funções estão automaticamente disponíveis.

Um ponto importante da dinâmica do Pyhton é a existência de uma convenção sobre identificadores (nomes de variáveis, classes, métodos etc…) circundados por dois underscores. Eles se referem aos identificadores especiais normalmente providos pelo próprio Python, ou seja, funções nativas.

Iremos usar o interpretador para inspecionar melhor esse módulo e conhecer melhor a linguagem.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Vamos começar pela função ***dir().*** Com ela, podemos listar todos os membros do escopo atual (sem parâmetros) ou de um determinado objeto.

```
builtins

dir()

pi = 3.1415
dir(pi)

dir(__builtins__)
```

Figura 1 – Builtins
Fonte: Elaborado pelo autor (2021)

Execute os comandos e avalie o retorno das funções do ***__builtins__.***

A partir de agora, vamos aprender a utilizar a linguagem para entendê-la.

Como vimos, o ***__builtins__*** tem mais de 150 membros. Não é produtivo conhecê-los todos agora, por isso, vamos focar nos mais importantes, principalmente os que nos ajudam a entender melhor a linguagem.

Função ***help()***, sem nenhum parâmetro, faz o interpretador entrar em modo de help, o que nos mostrará qualquer ajuda relacionada ao que for digitado. Porém, ao receber um parâmetro, a função ***help()*** mostrará o help associado ao objeto.

```
help(dir)

Help on built-in function dir in module builtins:

dir(...)
    dir([object]) -> list of strings

    If called without an argument, return the names in the current scope.
    Else, return an alphabetized list of names comprising (some of) the attributes
    of the given object, and of attributes reachable from it.
    If the object supplies a method named __dir__, it will be used; otherwise
    the default dir() logic is used and returns:
      for a module object: the module's attributes.
      for a class object:  its attributes, and recursively the attributes
        of its bases.
      for any other object: its attributes, its class's attributes, and
        recursively the attributes of its class's base classes.
```

Figura 2 – Help(dir)
Fonte: Elaborado pelo autor (2021)

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
help(print)

Help on built-in function print in module builtins:

print(...)
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

    Prints the values to a stream, or to sys.stdout by default.
    Optional keyword arguments:
    file:  a file-like object (stream); defaults to the current sys.stdout.
    sep:   string inserted between values, default a space.
    end:   string appended after the last value, default a newline.
    flush: whether to forcibly flush the stream.
```

Figura 3 – Help(print)
Fonte: Elaborado pelo autor (2021)

Função ***type()*** é nossa primeira função que tem parâmetros obrigatórios. Ela irá retornar o tipo/classe a que pertence o objeto usado como parâmetro.

```
type()

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-15-ff6dcb82abed> in <module>()
----> 1 type()

TypeError: type() takes 1 or 3 arguments

SEARCH STACK OVERFLOW
```

Figura 4 – Type() Error
Fonte: Elaborado pelo autor (2021)

A função ***type()*** exige 1 ou 3 parâmetros e, como não informamos nenhum, o Python levanta uma exceção do tipo ***TypeError***. Apesar de o nome da exceção remeter ao da função, nesse caso, é apenas uma coincidência. Essa exceção ocorre sempre que parâmetros obrigatórios não são informados.

É bom lembrar que, usando o interpretador, o retorno das expressões é automaticamente impresso no console, tornando desnecessário o uso do ***print()***.

Já quando estamos usando uma função built-in no editor você precisará usar o **print()** também para exibir os resultados. Por exemplo, para usar a função len() para obter o comprimento de uma string, você pode escrever o seguinte código:

```
string = "Olá, mundo!"
tamanho = len(string)
print(tamanho)
```

Código-fonte 1 – Uso das funções len e print
Fonte: Elaborado pela autora (2022)

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Em resumo, em Python, as funções built-in são funções pré-definidas e incorporadas na linguagem. Elas estão disponíveis para uso imediato assim que você inicia o interpretador Python ou executa o seu código.

Algumas das funções built-in mais usadas são:

- **print()** - Imprime uma string ou uma representação de um objeto na saída padrão.
- **len()** - Retorna o comprimento de uma string, lista, dicionário etc.
- **str()** - Converte um objeto em uma string.
- **int()** - Converte uma string ou outro objeto em um inteiro.
- **float()** - Converte uma string ou outro objeto em um número de ponto flutuante.
- **sum()** - Retorna a soma dos elementos de uma lista de números.
- **min()** - Retorna o menor elemento de uma lista de números.
- **max()** - Retorna o maior elemento de uma lista de números.
- **sorted()** - Retorna uma lista ordenada a partir de uma lista desordenada.
- **reversed()** - Retorna uma lista invertida a partir de uma lista original.
- **range()** - Retorna uma sequência de números, começando de 0 (ou de um valor especificado) até um determinado valor.
- **enumerate()** - Adiciona um contador a uma lista ou outro objeto iterável.
- **zip()** - Agrupa elementos de duas ou mais listas e retorna um objeto zip.
- **map()** - Aplica uma função a cada elemento de uma lista e retorna um objeto map.
- **filter()** - Filtra elementos de uma lista com base em uma função booleana e retorna um objeto filter.

Observe que algumas funções built-in podem requerer argumentos obrigatórios e/ou argumentos opcionais. A documentação do Python fornece informações detalhadas sobre como usar cada uma das funções built-in, incluindo a lista de argumentos, o que eles fazem e o que a função retorna.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

1.2 Minha primeira função

As funções são um dos conceitos fundamentais da matemática e da programação. Em matemática, uma função é uma relação entre duas coleções de elementos, onde a cada elemento de uma coleção (chamada de domínio) corresponde exatamente um e somente um elemento da outra coleção (chamada de imagem).

Na programação, as funções são construções que permitem encapsular um pedaço de lógica e reutilizá-lo mais tarde. Elas consistem em uma série de instruções que são executadas quando a função é chamada. As funções recebem zero ou mais entradas (chamadas de argumentos ou parâmetros) e retornam uma ou mais saídas. Elas ajudam a organizar o código, a torná-lo mais legível e a facilitar a manutenção.

Para entender como criar uma função e como ela funciona, vamos primeiramente escrever aquele algoritmo de velocidade média que tínhamos pensado:

```
#solicitar distância e tempo
distancia = float(input("Digite a distância percorrida"))
tempo = float(input("Digite o tempo da viagem"))

#calcular a velocidade média
velocidade_media = distancia/tempo

#exibir o resultado
print("A velocidade média é {} km/h".format(velocidade_media))
```

Código-fonte 2 – Algoritmo de velocidade média sem função
Fonte: Elaborado pelo autor (2020)

O que faremos agora é indicar ao Python que queremos criar uma nova função. Isso é feito por meio da palavra reservada “def”, que exige que indiquemos um nome para nossa função, seguido de parênteses:

```
def calcularVelocidadeMedia():
```

Código-fonte 3 – Criando uma nova função em Python
Fonte: Elaborado pelo autor (2020)

Uma vez que nossa função está criada, vamos pegar todo o código do nosso algoritmo e colocar dentro dela:

```
#Função que calcula a velocidade média
def calcularVelocidadeMedia():
    #solicitar distância e tempo
    distancia = float(input("Digite a distância percorrida"))
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
        tempo = float(input("Digite o tempo da viagem"))
        #calcular a velocidade média
        velocidade_media = distancia/tempo
        #exibir o resultado
        print("A velocidade média é {} km/h".format(velocidade_media)))
```

Código-fonte 4 – Colocando o algoritmo dentro de uma função em Python
Fonte: Elaborado pelo autor (2020)

Se você tentar executar esse script, notará que nada acontece! Isso ocorre porque uma função só entra em funcionamento quando é convocada. Por isso, fora da função e dentro do programa principal, faremos uma chamada para ela.

```
    #Função que calcula a velocidade média
    def calcularVelocidadeMedia():
        #solicitar distância e tempo
        distancia = float(input("Digite a distância percorrida"))
        tempo = float(input("Digite o tempo da viagem"))
        #calcular a velocidade média
        velocidade_media = distancia/tempo
        #exibir o resultado
        print("A velocidade média é {} km/h".format(velocidade_media))


    #aqui começa o programa principal
    calcularVelocidadeMedia()
```

Código-fonte 5 – Chamada para a função criada em Python
Fonte: Elaborado pelo autor (2020)

Pronto! Não só criamos nossa função como aprendemos a fazer uma chamada. Agora, toda vez que quisermos calcular a velocidade média, bastará escrevermos “calcularVelocidadeMedia()”.

Mas não vá embora ainda, pois temos alguns pontos para melhorar!

1.3 O poder dos parâmetros

Os parâmetros são uma parte importante das funções em Python, pois permitem que você defina funções que possam ser reutilizadas com diferentes valores. Quando você define uma função, você pode especificar um ou mais

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

parâmetros que serão usados dentro da função. Quando a função é chamada, os valores para esses parâmetros são passados como argumentos.

Por exemplo, na nossa função de velocidade média, poderíamos fazer:

```
#Função que calcula a velocidade média
def calcularVelocidadeMedia(distancia, tempo):
    #calcular a velocidade média
    velocidade_media = distancia/tempo
    print("A velocidade média é {} km/h".format(velocidade_media))
#aqui começa o programa principal
dist = float(input("Informe a distância: "))
temp = float(input("Informe o tempo: "))
#chamando a função com valores definidos pelo usuário
calcularVelocidadeMedia(dist, temp)
```

Código-fonte 6 – Modificando a função para receber parâmetros
Fonte: Elaborado pela autora (2023)

Nessa versão da função, o código de entrada de dados foi removido e os parâmetros distância e tempo são passados diretamente para a função.

Além disso, é possível definir valores padrão para os parâmetros, para que a função possa ser chamada sem passar valores para todos os parâmetros. Se um valor padrão é especificado para um parâmetro, ele será usado se nenhum valor for passado para esse parâmetro quando a função é chamada.

```
def calcularVelocidadeMedia(distancia, tempo=1):
    velocidade_media = distancia / tempo
    print("A velocidade média é {} km/h".format(velocidade_media))
#aqui começa o programa principal
dist = float(input("Informe a distância: "))
#chamando a função com valores definidos pelo usuário
calcularVelocidadeMedia(dist)
```

Código-fonte 7 – Modificando a função para receber parâmetros
Fonte: Elaborado pela autora (2023)

Nessa versão da função, o tempo padrão é definido como 1, o que significa que, se não for fornecido um tempo específico, a função usará 1 hora como tempo. Note a mudança da chamada função onde passamos apenas a distância.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

1.4 Alô!? Me responde função! - retorno de valores

Existem cenários nos quais desejamos que uma função execute uma tarefa que não gere dados ao final, como fechar uma conexão com o banco de dados ou remover um arquivo do HD. Em outras ocasiões, porém, o objetivo da nossa função é retornar alguma informação valiosa, como no caso da nossa função que calcula a velocidade média.

Se prestarmos bastante atenção à nossa função, ela "amarra" seu funcionamento a um cenário no qual vamos sempre "printar" o resultado na tela. Mas e se quiséssemos apenas gravar esse valor em um arquivo ou se quiséssemos utilizá-lo em outro cálculo?

Quando nos vemos diante de uma função que tem por objetivo retornar um dado, devemos abrir mão do uso de "prints" dentro da função e utilizar o comando return.

O comando return faz com que uma função seja encerrada e que um determinado valor seja devolvido para o local onde ocorreu a chamada da função. Ficou confuso? Então vamos devagar.

Nossa primeira tarefa é fazer o print desaparecer e substituí-lo por um return:

```
#Função que calcula a velocidade média
def calcularVelocidadeMedia(distancia, tempo):
    #calcular a velocidade média
    velocidade_media = distancia/tempo
    #retornar o resultado
    return velocidade_media
```

Código-fonte 8 – Modificando a função para retornar um valor
Fonte: Elaborado pelo autor (2020)

Agora, precisamos adequar a chamada da função. Afinal de contas, será retornado um valor com o qual precisaremos lidar. Portanto:

```
#Função que calcula a velocidade média
def calcularVelocidadeMedia(distancia, tempo):
    #calcular a velocidade média
    velocidade_media = distancia/tempo
    #exibir o resultado
    return velocidade_media

#aqui começa o programa principal
dist = float(input("Informe a distância"))
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
     temp = float(input("Informe o tempo"))
     #chamando a função com valores definidos pelo usuário
     print("A velocidade media é
{}".format(calcularVelocidadeMedia(dist, temp)))
```

Código-fonte 9 – Modificando a chamada da função que retorna um valor
Fonte: Elaborado pelo autor (2020)

O uso de funções é um grande diferencial para qualquer desenvolvedor, e uma ótima forma de exercitar é refazendo todos os algoritmos que você concluiu até hoje adequando a esse formato.

1.5 1, 2, 3, N parâmetros

Em Python, podemos definir uma função para receber uma quantidade variável de parâmetros, utilizando o asterisco (*) antes do nome do parâmetro. Isso significa que a função pode receber uma quantidade indefinida de argumentos, que serão agrupados em uma tupla.

Dessa forma, podemos chamar a função passando uma quantidade arbitrária de argumentos, sem ser limitados pela quantidade de parâmetros que foi definida na função. Esse recurso é útil em situações onde não sabemos ao certo quantos argumentos serão necessários ou quando queremos permitir que o usuário passe vários argumentos adicionais.

```
     #Criando a função de saudação
     def fSaudacao(pPeriodo, *pNome):
      for i in pNome:
         if pPeriodo.lower() in ('manha', 'm'):
           print(f'Bom dia, {i}! Como vai?')
         elif pPeriodo.lower() in ('tarde', 't'):
           print(f'Boa tarde, {i}! Como vai?')
         else:
           print(f'Boa noite, {i}! Como vai?')
     # #aqui começa o programa principal
     fSaudacao('n', 'João', 'Maria', 'Pedrinho')
```

Código-fonte 10 – Função de saudação com número de parâmetros variáveis
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

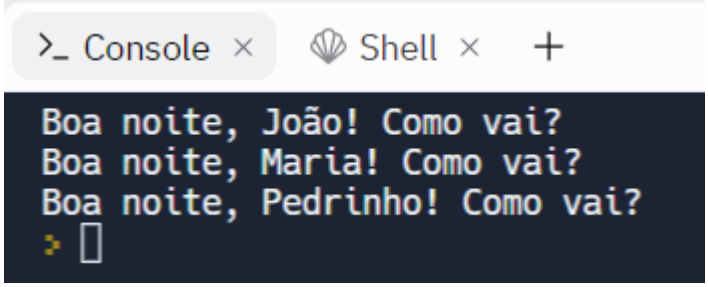

Figura 5 – Saída da Saudação com número de parâmetros variáveis
Fonte: Elaborado pela autora (2023)

2 FUNÇÕES INLINE

Imagine que você está desenvolvendo um aplicativo para uma empresa que precisa calcular o salário dos funcionários baseado em suas horas trabalhadas. Em vez de criar uma função separada para realizar essa tarefa, você pode usar uma função inline que fará tudo de uma só vez, diretamente no seu código.

Vamos caminhar nesse assunto?

2.1 É patins? Grego? Funções INLINE ou Lambda

As funções inline são uma forma de escrever pequenas funções de uma única linha dentro do próprio código, sem precisar escrevê-las em uma função separada. Em Python, elas são conhecidas como funções anônimas e são criadas usando a palavra-chave **lambda**. Por isso esse nome é mais conhecido.

Além de tornar o seu código mais limpo e organizado, as funções inline também ajudam a melhorar a performance do aplicativo, uma vez que não há a necessidade de chamar uma função separada a cada vez que o cálculo precisar ser realizado.

A sintaxe de uma função lambda é a seguinte:

```
lambda arguments : expression
```

Código-fonte 11 – Estrutura básica do lambda
Fonte: Elaborado pelo autor (2022)

Onde arguments são os parâmetros da função e expression é a expressão que será executada quando a função for chamada.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

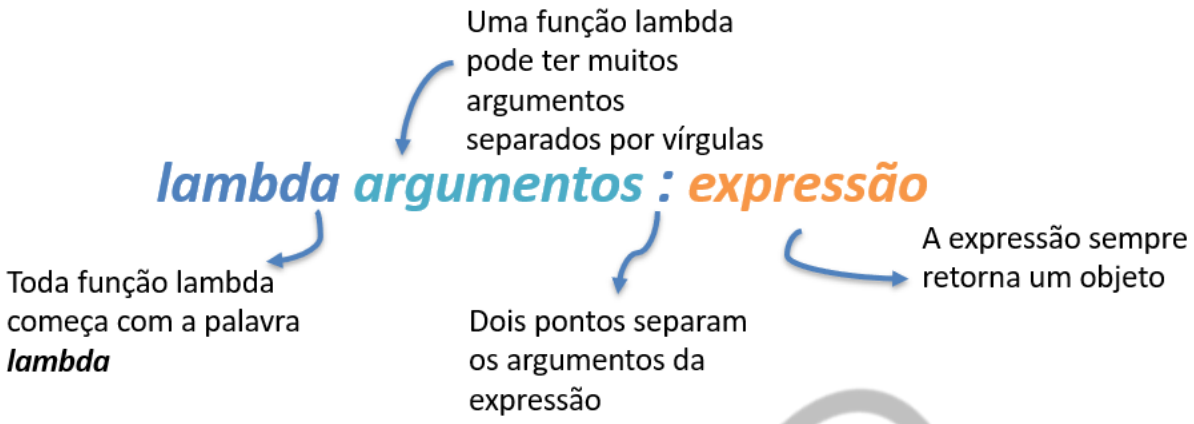

Figura 6 – Como usar a função lambda
Fonte: Elaborado pela autora (2023)

Neste exemplo, criamos uma função lambda chamada soma, que recebe dois argumentos a e b e retorna a soma desses dois valores. Em seguida, chamamos a função soma passando como argumentos 1 e 2 e obtemos o resultado 3. Note ainda que o “return” da função lambda é implícito.

```
soma = lambda a, b : a + b
print(soma(1, 2))
# Saída: 3
```

Código-fonte 12 – Soma de 2 números com lambda
Fonte: Elaborado pela autora (2023)

Podemos ainda usar funções lambda com desvios condicionais, onde a estrutura é um pouco mais complexa podendo ser escritas com a utilização de desvios condicionais, como o if e o else. Dessa forma, podemos fazer uma verificação e, dependendo do resultado, executar uma ação diferente.

A sintaxe básica de funções lambda com desvios condicionais é a seguinte:

```
lambda argumentos: expressão if condição else outra_expressão
```

Código-fonte 13 – Estrutura básica do lambda
Fonte: Elaborado pelo autor (2022)

Nessa estrutura, os argumentos são um ou mais argumentos que a função lambda irá receber, condição é a condição a ser avaliada, e expressão e outra_expressão são duas expressões diferentes que serão retornadas dependendo se a condição é verdadeira ou falsa.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

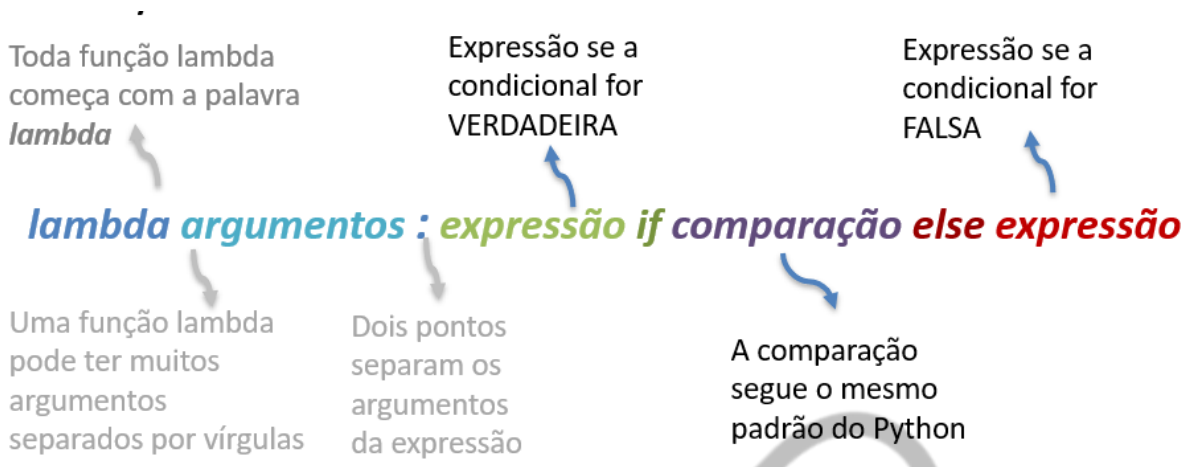

Figura 7 – Como usar a função lambda com desvios condicionais
Fonte: Elaborado pela autora (2023)

Por exemplo, podemos criar uma função lambda que, dado um número, verifica se ele é positivo ou negativo. Em caso de ser positivo, a função retorna "Positivo", e em caso de ser negativo, retorna "Negativo".

```
verificar_positivo = lambda x: "Positivo" if x >= 0 else "Negativo"
print(verificar_positivo (-1))
# Saída: Negativo
```

Código-fonte 14 – Positivo ou Negativo com lambda
Fonte: Elaborado pela autora (2023)

No geral, as funções lambda são úteis em situações onde precisamos de uma função simples e temporária, como por exemplo, como argumento em uma função que espera uma função como entrada, ou como retorno de outra função.

2.2 MAP

O map é uma função built-in do Python que tem como objetivo aplicar uma função a cada elemento de uma coleção, como uma lista, tupla, string, dicionário etc e retornar uma nova coleção com os resultados. É uma forma eficiente de aplicar uma operação a uma grande quantidade de elementos sem precisarmos escrever loops explícitos.

```
map(func, iter)
```

Código-fonte 15 – Estrutura básica do map
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

func é a função que será aplicada a cada elemento de **iter.** O resultado da chamada **map** é um objeto do tipo map. Para visualizar os resultados, é necessário aplicar a função **list** ou **tuple** ao objeto retornado por **map**, fazendo a conversão como no exemplo:

```
def dobro(x):
    return 2 * x
resultado = map(dobro, [1, 2, 3, 4, 5])
resultado = list(resultado)
print(resultado)
```

Código-fonte 16 – Map com função dobro
Fonte: Elaborado pela autora (2022)

A função dobro é aplicada a cada elemento da lista [1, 2, 3, 4, 5], e o resultado é armazenado no objeto resultado, que é convertido em lista antes de ser exibido.

E o casamento perfeito é quando usamos map com lambda. Note como o código fica enxuto. Modificando nosso exemplo

```
resultado = list(map(lambda x: 2 * x, [1, 2, 3, 4, 5]))
print(resultado)
```

Código-fonte 17 – Map com função lambda dobro
Fonte: Elaborado pela autora (2022)

3 A FRONTEIRA FINAL

Depois de aprendermos a lidar com funções, existe mais um espaço importantíssimo para concluir nossa caminhada rumo à modularização completa dos nossos programas: o uso de módulos.

Módulos são scripts Python que contêm funções e estruturas que podem ser incorporadas em outros scripts. Seremos capazes de utilizar nossas funções não apenas dentro de um mesmo script, mas entre diversos scripts que compõem um único sistema. Vamos lá?

3.1 Criando o módulo

Para começar nossa experiência, vamos partir do seguinte cenário: um script chamado *calc.py* (que conterá apenas as funções necessárias para as quatro

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

operações básicas) e um script chamado *teste.py* (que usaremos para testar o uso de módulos), sendo os dois criados dentro de um mesmo projeto.

No script *calc.py* vamos escrever nossas funções:

```
#função de soma
def somar(a, b):
    return float(a) + float(b)
#função de subtração
def subtrair(a, b):
    return float(a) - float(b)
#função de divisão
def dividir(a, b):
    if b==0:
        return 0
    return float(a) / float(b)

#função de multiplicar
def multiplicar(a, b):
         return float(a) * float(b)
```

Código-fonte 18 – Módulo calc.py
Fonte: Elaborado pelo autor (2020)

Pronto! Sem aprendermos nenhum comando novo, desenvolvemos nosso primeiro módulo! Resta-nos aprender a como utilizá-lo.

3.2 Utilizando o módulo

Para utilizarmos um módulo que foi criado, usaremos o comando chamado *import*. Com ele, seremos capazes de indicar o arquivo ao qual queremos incorporar funções e estruturas.

No nosso script chamado teste.py, teremos a seguinte linha:

```
#importação do módulo calc.py
import calc
```

Código-fonte 19 – Importação do módulo calc.py
Fonte: Elaborado pelo autor (2020)

Uma vez que fizermos a importação, o uso do módulo deverá ser feito obedecendo à estrutura: *nomedomodulo.nomedafuncao*. No nosso caso, podemos exemplificar assim:

```
#importação do módulo calc.py
import calc
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
#solicitando valores ao usuário
valor1 = input("Digite um valor: ")
valor2 = input("Digite outro valor: ")

#armazenando a soma
soma = calc.somar(valor1, valor2)
#exibindo a soma
print("A soma é {}".format(soma))
```

Código-fonte 20 – Uso da função soma do módulo calc.py
Fonte: Elaborado pelo autor (2020)

Se formos utilizar muitas funções de um determinado módulo, podemos fazer a importação de cada uma delas utilizando a seguinte estrutura:

```
#importação de funções específica do módulo calc.py
from calc import somar, subtrair
```

Código-fonte 21 – Importação de funções específicas do módulo calc.py
Fonte: Elaborado pelo autor (2020)

Dessa maneira, a chamada da função poderá ser feita sem a necessidade de escrever o nome do módulo e o sinal de ponto. Veja:

```
#importação de funções específica do módulo calc.py
from calc import somar, subtrair

#solicitando valores ao usuário
valor1 = input("Digite um valor: ")
valor2 = input("Digite outro valor: ")

#armazenando a soma
soma = somar(valor1, valor2)
#exibindo a soma
print("A soma é {}".format(soma))
```

Código-fonte 22 – Chamada de uma função especificada na importação do módulo calc.py
Fonte: Elaborado pelo autor (2020)

Ainda é possível importar todas as funções de uma só vez, mas esse recurso deve ser usado com cautela para que não tenhamos um programa desnecessariamente lento:

```
#importação de funções específica do módulo calc.py
from calc import *
```

Código-fonte 23 – Importação de todas as funções do módulo calc.py
Fonte: Elaborado pelo autor (2020)

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

4 HANDS ON: HORA DA PRÁTICA

4.1 Função é essencial

Um bom programador organiza seu código e prioriza reuso. Com o uso de funções iremos ganhar agilidade e legibilidade. Vamos a prática

4.1.1 Vai de pizza?

Vamos colocar a mão na massa pensando em pizza. Você é dono de uma pizzaria e está desenvolvendo um sistema de entrega para seus clientes. Cada cliente pode escolher o tamanho da pizza (pequena, média ou grande) e a quantidade de sabores (1, 2 ou 3). Seu sistema precisa calcular o preço final da pizza, levando em consideração o tamanho e a quantidade de sabores.

A pizza pequena custa R$20,00, a média custa R$30,00 e a grande custa R$40,00. Cada sabor adicional custa R$5,00.

Crie uma função chamada "calcula_pizza" que receba o tamanho e a quantidade de sabores da pizza e retorne o preço final.

```
def calcula_pizza(tamanho, quantidade_sabores):
  if tamanho.lower() == 'pequena':
    preco = 20
  elif tamanho.lower() == 'media':
    preco = 30
  else:
    preco = 40
  preco += (quantidade_sabores - 1) * 5
  return preco

#Teste da função
print(calcula_pizza('pequena', 1)) #20
print(calcula_pizza('media', 2)) #35
print(calcula_pizza('grande', 3)) #50
```

Código-fonte 24 – Calculando o preço da pizza
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

4.1.2 Viagem intergaláctica

Imagine que a equipe de Han Solo precisa escolher o próximo destino para a sua viagem através da galáxia. Eles têm uma lista de planetas que gostariam de visitar, mas querem filtrar apenas aqueles que tenham uma atmosfera respirável.

Dos planetas "Tatooine", "Hoth", "Endor", "Alderaan", "Naboo", somente "Hoth" não possui uma atmosfera respirável. Podemos usar a função map para aplicar a função lambda que verifica se o planeta tem atmosfera respirável ou não.

```
     verificar_atmosfera = lambda planeta: "Ufa, pode respirar
a vontade" if planeta != "Hoth" else "Cuidado! Não ouse retirar
a vestimenta"

     print(list(map(verificar_atmosfera, ["Tatooine", "Hoth",
"Endor", "Alderaan", "Naboo"])))
```

Código-fonte 25 – Calculando o preço da pizza
Fonte: Elaborado pela autora (2023)

As funções em Python são uma ferramenta essencial para escrever códigos mais organizados e de fácil manutenção permitindo dividir o código em partes menores e reutilizáveis, melhorando a legibilidade e facilitando a identificação de erros. Com a passagem de parâmetros, criamos possibilidade de adaptar o comportamento da função a diferentes situações. A evolução passa pelas funções Lambda que são uma maneira rápida e concisa de criar funções simples sem a necessidade de uma definição formal e junto com o Map, essas funções são poderosas ferramentas para manipular listas e conjuntos de dados de maneira eficiente. Aproveite para exercitar todo esse potencial.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

REFERÊNCIAS

PIVA, Dilermando Júnior. **Algoritmos e programação de computadores**. São Paulo: Elsevier, 2012.

PUGA, Sandra; RISSETTI, Gerson. **Lógica de programação e estrutura de dados**. São Paulo: Pearson Prentice Hall, 2009.

RAMALHO, Luciano. **Python Fluente**: Programação clara, concisa e eficiente. São Paulo: Novatec, 2015.

PYTHON; Et al. Documentação oficial do Python. 2023. Disponível em: <https://docs.python.org/3/>. Acesso em: 04 fev 2023.

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com